Water Network Design Using Genetic Algorithm, Simulated Annealing, and a Hybrid GA with Local Search and MIP Repair

Lacatus Stefania, Maximiuc Teodora

***Abstract*—This paper presents a project on water distribution network design. The goal is to choose pipe diameters from a fixed set of options so that the network is inexpensive while still satisfying hydraulic constraints. Three search methods were considered: a Genetic Algorithm (GA), Simulated Annealing (SA), and a hybrid Genetic Algorithm enhanced with Local Search and Mixed-Integer Programming (MIP) repair. All methods use the same network representation and rely on hydraulic simulation for evaluation, but they differ in the way candidate solutions are explored and refined. The implementation was tested on standard benchmark instances, and the report summarizes the design choices, the experimental setup, and the main comparison points between the three optimization strategies.**

***Index Terms*—water network design, genetic algorithm, simulated annealing, EPANET, benchmark optimization**

I. Introduction

Designing a water network is an optimization problem. Pipe diameters affect both cost and hydraulic performance, so a good solution must balance these two aspects. If diameters are too small, pressure may fall below the required level. If diameters are too large, the design becomes unnecessarily expensive.

In this project, three methods were tested: GA, SA, and a hybrid GA with Local Search and MIP repair. All methods generate candidate pipe-sizing solutions, evaluate them with the same hydraulic simulator, and try to improve them over time. Because the evaluation component is shared, the comparison focuses mainly on the search and refinement strategy used by each method.

II. Problem Formulation

A candidate solution is represented as an integer vector. Each position corresponds to a decision pipe, and the value stored in that position represents one diameter option from a finite catalog.

The optimization objective is modeled with the following fitness function:

$$F = C + P_h + P_v \tag{1}$$

where C is the total pipe installation cost, P_h is the penalty for pressure violations, and P_v is the penalty for velocity violations.

This formulation is practical because it allows both feasible and infeasible designs to be explored, while strongly penalizing solutions that do not satisfy hydraulic constraints.

The other cost method uses a multi-objective optimization approach:

- minimize total pipe cost;
- minimize pressure deficit;
- maximize network resilience.

The fitness score is defined as:

$$F(x) = (D(x),\ C(x) + P_h(x),\ -R(x))$$

where x represents a candidate pipe-diameter solution, $D(x)$ is the average pressure deficit, $C(x)$ is the total pipe cost, $P_h(x)$ is the pressure violation penalty, and $R(x)$ is the resilience index. Since NSGA-II is formulated as a minimization algorithm, resilience is written as $-R(x)$, so maximizing resilience becomes equivalent to minimizing negative resilience.

The pressure deficit is computed as:

$$D(x) = \frac{1}{|N|} \sum_{i \in N} \max\left(0,\ H_i^{\min} - H_i(x)\right)$$

where N is the set of network nodes, $|N|$ is the number of nodes, $H_i^{\min}$ is the minimum required pressure/head at node i, and $H_i(x)$ is the simulated pressure/head at node i for solution x.

A penalty is added to the cost when pressure constraints are violated. This allows infeasible solutions to still participate in the search but makes them less competitive.

III. Software Design

The implementation follows a simple modular structure.

First, a benchmark network is loaded from an EPANET input file. Then, a candidate solution is represented as an integer vector, where each position corresponds to one decision pipe and each value selects one diameter option from the available catalog.

A. Genetic Algorithm

The Genetic Algorithm maintains a population of candidate solutions. At each generation, better individuals are selected through tournament selection, new candidates are created by crossover, and diversity is maintained through mutation. Elitism is used so that the best solutions are preserved from one generation to the next.

GA was chosen because the pipe sizing problem is discrete, combinatorial, and highly nonlinear due to hydraulic simulation.

B. Simulated Annealing

Simulated Annealing starts from one solution and generates neighbors by modifying a small number of decision variables. A worse solution can still be accepted with a probability that depends on a temperature parameter. As the temperature decreases, the search becomes more conservative.

SA was chosen because it is simple, memory-efficient, and able to escape local optima during the early part of the search.

C. Hybrid Genetic Algorithm with Local Search and MIP Repair

A third method was implemented as an improved hybrid optimizer. It starts from a Genetic Algorithm framework, but adds two refinement mechanisms: Local Search (LS) and Mixed-Integer Programming (MIP) repair.

The population is evolved with a multi-objective GA based on NSGA-II. Each candidate solution is still an integer vector of pipe diameter choices, but solutions are evaluated using multiple criteria: hydraulic deficit, penalized cost, and resilience. This allows the algorithm to keep a diverse set of trade-off solutions instead of optimizing only one scalar score.

After each generation, a local improvement phase is applied to the elite part of the population. This Local Search modifies a small number of pipe decisions and keeps changes that improve the fitness. In this way, promising solutions are refined without relying only on crossover and mutation.

In addition, the best individuals are passed through a MIP-based repair step. The repair model considers a restricted neighborhood around the current pipe diameters and tries to find a lower-cost or more feasible nearby design while respecting linearized pressure-related constraints. This makes the search more robust, because partially good solutions can be repaired instead of being discarded.

This hybrid approach combines the exploration ability of GA, the fine-tuning ability of Local Search, and the constraint-handling ability of mathematical programming. As a result, it is more suitable for large and difficult benchmark instances than a plain GA or SA.

IV. Design Choices

The code was designed so that both GA and SA use the same solution representation and the same hydraulic evaluator. This was done to make the comparison fair and to avoid repeating the same logic in multiple places.

A solution is stored as an integer vector, where each value selects one diameter from a fixed catalog. This representation is appropriate because the problem is discrete and because both algorithms can easily modify integer values.

The evaluator is shared by both GA and SA methods. It loads the network, applies the chosen diameters, runs the hydraulic simulation, and computes the final fitness. The fitness combines cost with penalty terms for hydraulic violations. This makes the search more robust, since infeasible solutions can still be explored but are strongly penalized.

For the GA and LP hybrid method, the design was extended in two important ways. First, the optimization became multi-objective, so the search could consider not only cost and hydraulic deficit, but also a resilience-related measure. Second, the best candidate solutions were refined using two additional procedures: a Local Search phase for neighborhood improvement and a MIP-based repair phase for feasibility-oriented adjustment. These additions were introduced to improve convergence and robustness on larger benchmark networks.

V. Benchmark Instances

The implementation was tested on the following benchmark instances: `twoloop`, `newyork`, `goyang`, `balerma`, and `hanoi`.

Different instances use slightly different design modes: *replacement*, *zero_diameter_only*, and *parallel_expansion*. These modes were included because the benchmark files are not identical in structure.

VI. Implementation

The project was implemented in Python. The benchmark networks are read from EPANET `.inp` files, and hydraulic simulations are performed through the `wntr` library. The implementation is organized around one shared evaluation component and three search strategies.

A candidate solution is encoded as a vector of integers. Each integer represents one choice from a predefined diameter catalog. This encoding was used because it is compact, easy to modify, and works naturally for both GA and SA.

For every candidate solution, the program follows the same evaluation process:

1) load the benchmark network;
2) apply the candidate diameter choices to the decision pipes;
3) run the hydraulic simulation;
4) compute the total pipe cost;
5) compute penalties for pressure and velocity violations;
6) return the final fitness value.

The fitness function has the form

$$F = C + P_h + P_v \tag{2}$$

where C is the pipe cost, P_h is the pressure penalty, and P_v is the velocity penalty. This design makes the implementation robust, because infeasible solutions are not discarded immediately; instead, they receive a worse score and remain available for comparison.

A. Genetic Algorithm

In the code, GA works with a population of integer vectors. Each vector is one candidate solution, and each value is the index of a diameter from the catalog.

A solution looks like:

```
[11, 6, 9, 0, 8, 6, 6, 0]
```

The algorithm:

- creates an initial population,

- evaluates every individual with the shared network model,
- selects parents using tournament selection,
- creates children with crossover,
- changes some genes through mutation,
- keeps the best individuals through elitism.

The important part is that GA itself does not compute pressures or velocities. It only builds candidate vectors. The hydraulic simulation is always done by the evaluator.

B. Pseudo-code for Genetic Algorithm

Algorithm 1 Genetic Algorithm for Water Network Design

Require: Network model M, population size N, generations G, crossover rate p_c, mutation rate p_m, elite count E
Ensure: Best solution found

Initialize population P with N candidate solutions
Evaluate each individual in P using the hydraulic simulator
Store the best individual found so far
for $g = 1$ to G **do**
    Sort population by fitness
    Copy the best E individuals into the next population P_{new}
    while $|P_{\text{new}}| < N$ **do**
        Select parent$_1$ by tournament selection
        Select parent$_2$ by tournament selection
        if random number $< p_c$ **then**
            Apply crossover to parent$_1$ and parent$_2$
            Generate child$_1$, child$_2$
        else
            Copy parent$_1$, parent$_2$ as child$_1$, child$_2$
        end if
        Apply mutation to child$_1$ with rate p_m
        Apply mutation to child$_2$ with rate p_m
        Add child$_1$ and child$_2$ to P_{new}
    end while
    Replace P with P_{new}
    Evaluate each individual in P
    Update the best solution if a better individual is found
end for
return best solution

C. Simulated Annealing

In the code, SA uses the same solution format as GA, but it works with only one current solution instead of a population.

It starts from one initial vector, for example:

```
[11, 11, 11, 11, 11, 11, 11, 11]
```

Then, at each step, it creates a nearby solution by changing a few positions:

```
[11, 10, 11, 11, 9, 11, 11, 11]
```

If the new solution is better, it is accepted. If it is worse, it may still be accepted depending on the temperature. As the temperature decreases, the search becomes more conservative.

Like GA, SA uses the same evaluator for cost and hydraulic penalties. The difference is only in the search process: GA explores many solutions at once, while SA improves one solution step by step.

D. Pseudo-code for Simulated Annealing

Algorithm 2 Simulated Annealing for Water Network Design

Require: Network model M, initial temperature T_0, cooling factor α, number of iterations K
Ensure: Best solution found

Generate an initial solution s
Evaluate s using the hydraulic simulator
Set current solution $s_{\text{curr}} \leftarrow s$
Set best solution $s_{\text{best}} \leftarrow s$
Set temperature $T \leftarrow T_0$
for $k = 1$ to K **do**
    Generate a neighbor solution s'
    Evaluate s'
    $\Delta \leftarrow f(s') - f(s_{\text{curr}})$
    if $\Delta \leq 0$ **then**
        Accept s' as the new current solution
        $s_{\text{curr}} \leftarrow s'$
    else
        Accept s' with probability $\exp(-\Delta/T)$
        if s' is accepted **then**
            $s_{\text{curr}} \leftarrow s'$
        end if
    end if
    if $f(s_{\text{curr}}) < f(s_{\text{best}})$ **then**
        Update best solution: $s_{\text{best}} \leftarrow s_{\text{curr}}$
    end if
    Update temperature: $T \leftarrow \alpha T$
end for
return best solution

E. Hybrid GA + Local Search + MIP Repair

The code also includes a hybrid optimization method that extends the Genetic Algorithm.

In this implementation, the GA is multi-objective. Instead of ranking solutions only by one scalar fitness, the algorithm uses three components:

- mean pressure deficit,
- penalized total cost,
- negative resilience.

This means the search tries to minimize hydraulic deficit and cost while maximizing resilience.

The evolutionary part uses an NSGA-II style selection mechanism. The population is initialized randomly, offspring are generated by two-point crossover and integer mutation, and the next generation is selected from the combined parent-offspring population.

After the normal GA step, two refinement procedures are applied to elite individuals.

Local Search. A small neighborhood search is applied to promising solutions. At each step, one pipe decision is modified by changing the selected diameter index up or down. If the new solution dominates the previous one in the multi-objective sense, it is accepted. This improves exploitation near good solutions.

MIP Repair. A second refinement stage formulates a reduced mixed-integer optimization problem around the current design. Only nearby diameter options are considered for each pipe, which keeps the model compact. The objective is to minimize construction cost while satisfying linearized pressure-related constraints based on the current hydraulic state. If a repaired solution is found, it replaces the old one.

This hybrid strategy keeps the representation simple, because the chromosome is still an integer vector, but it adds stronger improvement mechanisms on top of the basic evolutionary search.

F. Design Modes

The code supports different ways of interpreting the benchmark files.

Replacement. The original pipes are directly resized. One gene changes one existing pipe.

Zero-diameter-only. Only the special candidate pipes with almost zero diameter are optimized. This is used for the New York benchmark.

Parallel expansion. The original network stays unchanged, and the code may add an extra parallel pipe. A zero value means no extra pipe is added. This was used for Hanoi.

VII. Results

A. Observations

The Two Loop instance produced feasible solutions with the Genetic Algorithm, with minimum pressure values slightly above the required threshold and zero penalty. This suggests that the chosen formulation is suitable for smaller benchmark networks.

The Hanoi instance was more difficult. Under a direct replacement formulation, the problem appeared infeasible with the chosen catalog, so a parallel expansion mode was also considered. This should be interpreted carefully, since it changes the modeling assumptions.

B. GA vs. SA Discussion Space

The following points can be used for the final comparison: which method found the lowest-cost feasible design, which method converged faster, which method was more robust across repeated runs, and which method handled larger instances better.

VIII. Discussion

A. Results Interpretation

The experimental results show that the performance of an optimization method depends strongly on the size and structure of the benchmark network.

For the smaller benchmark instances, such as `twoloop`, both GA and SA were able to produce feasible solutions. This indicates that the basic representation and evaluation model are appropriate for standard discrete pipe-sizing problems. In such cases, both classical metaheuristics can be effective, and the main difference is usually in convergence behavior and final cost.

For `goyang`, both classical methods again produced feasible solutions. This suggests that the problem is manageable for both a population-based strategy and a single-solution trajectory strategy. The difference between them is therefore more related to search efficiency and solution refinement than to basic feasibility.

The `newyork` and `hanoi` cases highlight the importance of modeling assumptions and search strength. In the original GA/SA experiments, some formulations remained infeasible or required a modified design mode such as parallel expansion. This shows that the benchmark itself may be difficult under strict direct replacement assumptions. In contrast, the hybrid method was able to achieve a 100% feasibility rate on both `newyork` and `hanoi` in the reported multi-run evaluation, which suggests that local refinement and repair mechanisms significantly improve robustness on difficult instances. :contentReference[oaicite:4]index=4

The largest instance, `balerma`, is the most informative in terms of scalability. Large networks increase the number of decision variables and make the hydraulic search landscape more complex. In the hybrid experiments, `balerma` remained challenging, but the method still achieved a 90% feasibility rate over repeated runs, which is a strong result for such a large benchmark. The runtime, however, increased substantially, showing that the improvement in robustness comes with a computational cost. :contentReference[oaicite:5]index=5

B. GA, SA, and Hybrid Method Comparison

The comparison between the three methods shows that each one has a different strength.

The standard GA is effective for exploring many candidate solutions at once. It is well suited to discrete pipe-sizing problems because crossover and mutation can create diverse combinations of diameter choices. On smaller and medium benchmark instances, GA can produce feasible solutions with good cost values. However, as the size of the network increases, GA may require many evaluations before it converges to a strong feasible design.

SA has a simpler search structure because it follows one current solution and improves it step by step. This makes it memory-efficient and easier to implement. It can perform well when the neighborhood structure is informative and when local improvements are sufficient to move toward a good design. However, because SA explores only one trajectory at a time, it may be less diverse than GA and more sensitive to the initial solution and cooling schedule.

The hybrid GA + Local Search + MIP Repair combines exploration and refinement. Like GA, it maintains a population and can explore multiple parts of the search space

TABLE I
BENCHMARK RESULTS FOR GA AND SA WITH COST

Instance	Method	Cost	Min Pressure	Max Velocity	P Penalty	V Penalty	Status
Two Loop	GA	420000	30.813591	2.019671	0	0	Feasible
Two Loop	SA	465000	30.521561	1.563673	0	0	Feasible
New York	GA	50509810.08	30.500391	2.516735	59781712500000	0	Infeasible
New York	SA	52490827.20	30.500429	2.516735	59781706250000	0	Infeasible
Goyang	GA	100562	15.624146	1.266111	0	0	Feasible
Goyang	SA	97621	15.466773	1.055376	0	0	Feasible
Hanoi (replacement)	SA	10214700	19.619724	6.831937	193101086425.78125	0	Infeasible
Hanoi (parallel)	GA	480200	30.085644	3.415969	0	0	Feasible
Hanoi (parallel)	SA	428550	30.005691	4.649981	0	0	Feasible
Balerma	GA	46851894	21.019451	2.771891	0	122683.241963	Pressure feasible
Balerma	GA	29010739	20.342272	2.982037	0	0	Feastable

TABLE II
BENCHMARK RESULTS FOR GA AND SA USING THE SAME MUSD COST MODEL

Instance	Method	Cost (MUSD)	Min Pressure	Max Velocity	P Penalty	V Penalty	Status
Two Loop	GA	1.512757	30.525442	1.896688	0.000000	0.000000	Feasible
Two Loop	SA	1.871512	30.887188	1.534974	0.000000	0.000000	Feasible
New York	GA	2.865921	30.500290	2.516735	16435.287520	0.000000	Infeasible
New York	SA	4.188564	30.500423	2.516735	16435.104117	0.000000	Infeasible
Goyang	GA	0.182831	15.466773	1.055376	0.000000	0.000000	Feasible
Goyang	SA	0.276932	15.624146	1.109844	0.000000	0.000000	Feasible
Hanoi	GA	1.957059	29.995691	4.649981	0.000093	0.000000	Infeasible
Hanoi	SA	16.134643	38.712318	5.881845	0.000000	0.000000	Feasible
Balerma	GA	40.515975	21.987543	2.875238	0.000000	0.178801	Infeasible
Balerma	SA	131.231283	21.360102	3.164122	0.000000	0.839822	Infeasible

TABLE III
BENCHMARK RESULTS FOR THE HYBRID GA + LOCAL SEARCH + MIP REPAIR METHOD

Instance	Best Cost	Mean Cost	Std Cost	Best Deficit	Feasibility Rate	Avg Runtime (s)	Avg Pareto Size	Avg Feasible Pareto Size
Two Loop	1.543	1.655	0.118	0.0000	100.0%	88.93	47.30	21.20
New York	278.744	331.111	42.683	0.0000	100.0%	728.79	107.10	8.80
Goyang	0.228	0.255	0.023	0.0000	100.0%	388.66	92.90	16.80
Hanoi	25.395	28.984	3.946	0.0000	100.0%	546.05	82.40	3.10
Balerma	33.868	39.919	2.672	0.0000	90.0%	14571.63	245.80	29.60

simultaneously. Unlike the classical GA, however, it also improves elite solutions through a local neighborhood search and attempts structured feasibility-oriented repair through a reduced MIP model. This gives it an important advantage on difficult benchmark instances, because partially good solutions do not need to be discarded; they can be refined into better ones.

The hybrid method also provides richer output than the single-objective GA and SA implementations. Because it is multi-objective, it does not return only one candidate design, but a set of trade-off solutions between deficit, cost, and resilience. This is useful in practice, since a designer may prefer either the cheapest feasible solution or a slightly more expensive but hydraulically safer design.

IX. CONCLUSION

This project studied three strategies for discrete water distribution network design: a Genetic Algorithm, Simulated Annealing, and a hybrid Genetic Algorithm with Local Search and MIP repair.

The experiments show that both GA and SA can solve standard benchmark instances and provide a useful baseline comparison between population-based and trajectory-based metaheuristics.

The hybrid method extends the GA framework in a meaningful way. By combining multi-objective search with local improvement and structured repair, it achieves higher robustness and better feasibility on difficult benchmark instances. In the reported experiments, the hybrid method achieved full feasibility on most benchmarks and remained effective even on large instances such as `balerma`, although with substantially

higher runtime.

Overall, the study suggests that simple metaheuristics are useful for baseline experimentation, but hybrid optimization methods are more suitable when the benchmark is large, difficult, or sensitive to hydraulic constraints.